

ASKSAM: THE SAMAGAMA KNOWLEDGE HUB

PROJECT REPORT

TEAM LEAD: MANO SHRUTHI S

INDEX

1. Introduction and Motivation
 2. Problem Statement
 3. System Architecture & Database Understanding
 4. Methodology & Platform Workflows
 5. Implementation Details & Tech Stack
 6. Results and Discussions
 7. Conclusion
- Appendix

1. INTRODUCTION

AskSam is a collaborative Question-and-Answer (Q&A) platform developed for the Samagama community under the Vicharanashala Internship Program, IIT Ropar. The platform is designed to provide a centralized knowledge repository where students can ask questions, receive verified answers, and access previously resolved queries.

Key Highlights of AskSam

➤ Centralized Knowledge Base

- Stores academic, internship, and administrative information in a single location.
- Eliminates dependency on scattered communication channels such as WhatsApp and Discord.
- Makes verified information easily accessible to all users.

Example Questions:

- How can I apply for an NOC?
- When will internship certificates be issued?
- What is the deadline for project submission?

➤ Question Categorization System

- Questions are automatically organized into meaningful categories.
- Improves search efficiency and knowledge management.
- Helps users quickly navigate to relevant discussions.

➤ Categories and Examples:

CATEGORY	DESCRIPTION	EXAMPLE QUESTIONS (3 EACH)
 TECHNICAL QUERIES	Programming, frameworks, debugging, databases, tools, and software development related questions.	• How do I implement JWT authentication in NestJS? • Why am I getting a MongoDB connection error? • How can I integrate Socket.IO with React?
 INTERNSHIP RELATED QUERIES	Internship tasks, reports, evaluations, deadlines, and project submissions related questions.	• What are the internship evaluation criteria? • How should weekly reports be submitted? • What are the project submission deadlines?
 DOCUMENTATION & NOC QUERIES	Questions related to official documents, NOCs, certificates, offer letters, and verification procedures.	• How can I apply for an NOC? • When will offer letters be released? • How do I download my internship certificate?
 ZOOM SESSION RELATED QUERIES	Questions regarding online meetings, attendance, schedules, and recorded sessions.	• What is today's Zoom meeting link? • Where can I find recorded sessions? • When is the next mentorship session?
 COMMUNITY & VIBE RELATED QUERIES	Questions focused on student experiences, collaboration, feedback, and learning environment.	• How was today's session? • What skills are useful for this internship? • How can I collaborate with other interns?
 GENERAL ADMINISTRATIVE QUERIES	Questions related to announcements, deadlines, procedures, support contacts, and other administrative	• Who should I contact for support? • Are there any new announcements? • What are the important deadlines this month?

➤ Community-Driven Answering System

- Students and community members can contribute answers.
- Encourages collaborative learning and peer support.
- Multiple users can provide different perspectives and solutions.

Example Questions:

- Which technology stack should I use for my project?
- How did you solve the MongoDB connection issue?
- What resources helped you complete the assignment?

➤ Answer Verification Mechanism

- Submitted answers undergo administrative review.
- Only verified and approved answers become official responses.
- Prevents misinformation and improves trustworthiness.

Example Scenario:

- A student asks: "How can I obtain an NOC?"
- Community members submit answers.
- An administrator verifies the correct process.
- The approved answer becomes the official response.

➤ **Canonical FAQ Generation**

- Verified answers are automatically converted into permanent FAQs.
- Creates a continuously growing knowledge repository.
- Reduces repetitive questions from future users.

Example FAQs:

- NOC Application Procedure
- Internship Completion Certificate Process
- Weekly Report Submission Guidelines
- Project Evaluation Criteria

➤ **Advanced Search Functionality**

- Supports full-text search across questions and FAQs.
- Helps users find answers before posting duplicate questions.
- Improves information retrieval speed.

Example Searches:

- "NOC"
- "Offer Letter"
- "Zoom Recording"
- "Project Submission"

➤ **Real-Time Communication**

- Implemented using Socket.IO.
- Provides instant updates and notifications.
- Ensures users receive the latest information without refreshing the page.

Example Notifications:

- New answer added to your question.
- Your answer has been verified.
- A new FAQ has been published.
- Your question requires additional clarification.

➤ **Role-Based Access Control (RBAC)**

- Different permissions for Students, Community Members, and Administrators.
- Protects critical operations from unauthorized access.
- Maintains platform integrity and security.

➤ **Role Responsibilities:**

- Student → Search FAQs, ask questions, participate in discussions.
- Community Member → Submit answers and help peers.
- Administrator → Verify answers, manage FAQs, and moderate content.

➤ **Fault-Tolerant Architecture**

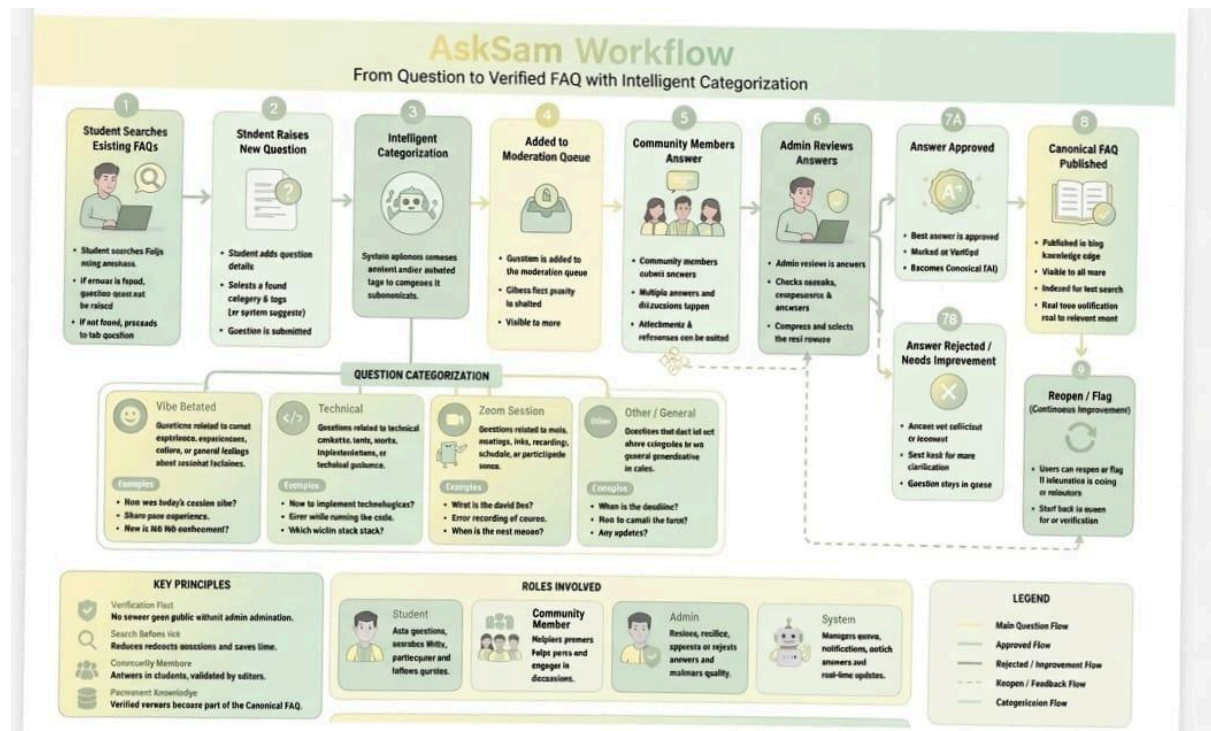
- MongoDB serves as the primary database.
- JSON fallback mechanism ensures continued operation during database failures.
- Enhances system reliability and availability.

Benefits:

- Improved uptime.
- Reliable data access.
- Better user experience during unexpected failures.

Purpose of AskSam

The primary purpose of AskSam is to transform temporary discussions into a permanent and searchable knowledge repository. By combining intelligent categorization, community participation, answer verification, and FAQ generation, the platform reduces repetitive queries, improves information accessibility, minimizes administrative workload, and ensures that students receive accurate and reliable information whenever required.



MOTIVATION

In fast-paced student communities, sharing information smoothly is always a challenge. Important announcements, links, and guidelines easily get lost under thousands of messages in chaotic chat groups. AskSam was created to transform these temporary, messy conversations into an organized, permanent, and verified knowledge base.

Why AskSam is Needed:

➤ Preventing Information Loss

- Important technical solutions and administrative updates get buried quickly in casual chat histories.
- Students waste time scrolling through old threads or asking the same questions repeatedly.
- AskSam provides a central repository where valuable community insights are permanently saved.
- *Real-World Scenario*: Important links or onboarding instructions shared weeks ago becoming impossible to find when needed.

➤ Stopping the Repeat Question Loop

- Mentors and student leads spend excessive time answering the exact same queries over and over again.
- AskSam implements a smart search tool that forces upfront discovery, helping users find answers before posting.
- *Real-World Scenario:* Multiple students constantly asking about daily Zoom links or project submission deadlines.

➤ Filtering Out Misinformation

- In open chat groups, unverified or outdated peer responses often create confusion.
- AskSam introduces a dedicated verification process where only admin-approved answers become official FAQs.
- *Real-World Scenario:* A student giving wrong information about NOC application procedures or project evaluation criteria.

➤ Eliminating Communication Delays

- Standard forum websites require users to refresh the page constantly to check for status updates.
- AskSam integrates real-time web sockets to push instant notifications and live updates to the user.
- *Real-World Scenario:* Waiting hours and manually reloading pages to check if an admin has reviewed a submitted answer.

➤ Ensuring System Reliability

- Traditional apps stop working entirely if the main database faces temporary network issues.
- AskSam features a smart local backup system (JSON fallback) to keep the platform accessible even during database failures
- *Real-World Scenario:* The primary database facing a brief timeout right before a critical project submission deadline.

➤ Ideate

To understand and organize the overall structure of the AskSam platform, a mind map was created during the ideation phase. The mind map provides a visual representation of the project's major components, including its purpose, key features, user roles, workflow, categories, technology stack, system architecture, benefits, and impact.



2. PROBLEM STATEMENT

2.1. The Primary Problem: Redundancy & Administrative Overload

2.1.1. Duplicate Questions at Scale

The defining characteristic of student queries is their high degree of repetition. A finite set of procedural questions recurs across every cohort with remarkable regularity. Large FAQ file results in ambiguity



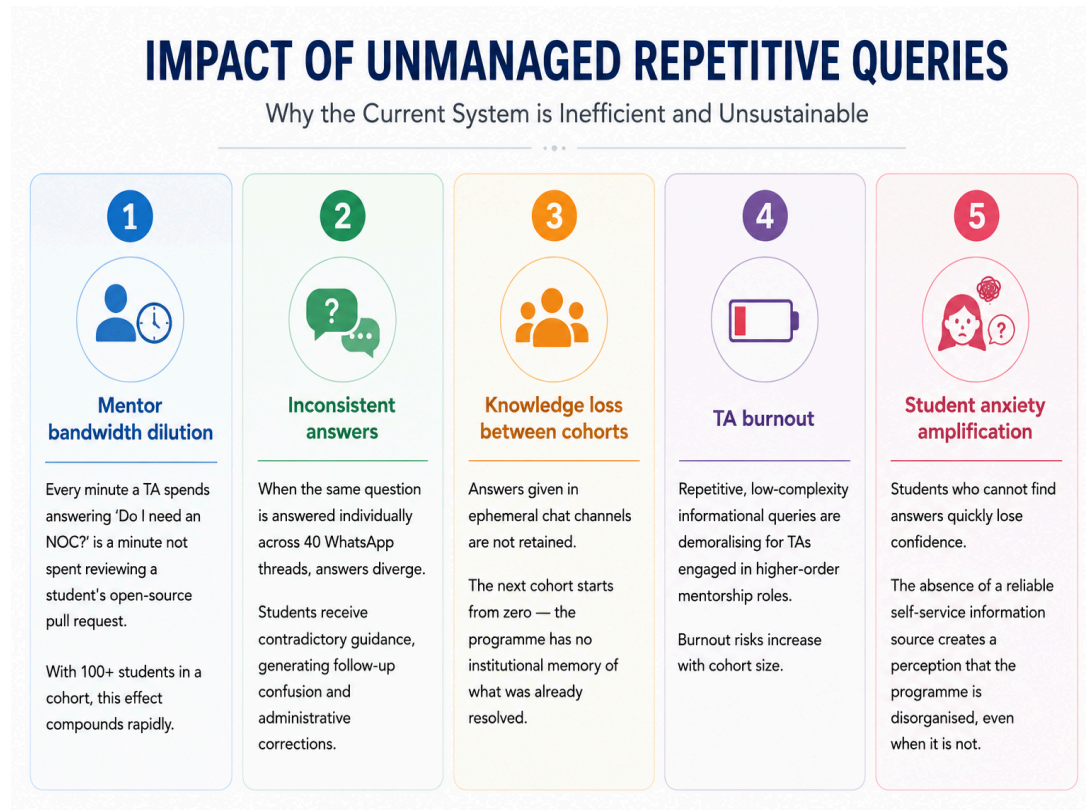
To avoid this we need analysis of the seeded FAQ corpus (faqData.json, 130+ entries) reve

Question Category	Representative Examples	Frequency Pattern
NOC (No Objection Certificate)	Do I need an NOC? What format? From whom?	Repeated every cohort by 30–50% of new joiners
Offer Letters	When is the offer letter issued? What does it contain?	Spikes at cohort start and programme end
Internship Timeline	When can I start? Can I defer? What is the hard deadline?	Persistent throughout registration window
Stipend & Payment	Is there a stipend? Is the internship paid?	Recurring pre-join question
Certificate Eligibility	What do I need to complete to get the certificate?	Repeated at Bronze–Silver transition
Leave & Attendance	Can I take leave for exams? What if I miss a day?	Spikes during exam seasons
Phase Progression	What are the badge phases? What is Gold/Platinum?	Recurring in orientation period

Each of these questions, when routed through informal channels, consumes TA admin time to answer individually — time that could be redirected to mentorship, code reviews, and programme improvement.

2.1.2. The True Cost of Unmanaged Repetition

The administrative cost of duplicate questions is not merely inconvenient — it is structurally harmful to programme quality.



A single well-structured, searchable answer to 'Do I need an NOC?' — visible to all students, permanently — saves hours of cumulative TA time per cohort cycle.

2.2. The Secondary Problem: Answer Quality & Trust

2.2.1. Peer Answers Are Not Inherently Reliable

A natural response to the repetitive-question problem is to crowdsource answers — to let students who know the answer help students who do not. However, unmoderated peer Q&A creates its own serious failure mode: incorrect answers propagate with the same confidence as correct ones.

Incorrect peer answers carry real consequences:

- A student told they do not need an NOC when they do may lose their internship position.

- A student misinformed about the programme deadline may submit work after the 31 December cutoff and forfeit their certificate.
- Contradictory answers about leave policy create confusion and appeals that consume additional admin time.

2.2.2. The Verification Gap

Without a formal verification mechanism, all peer answers exist in an epistemic grey zone — possibly correct, possibly outdated, possibly based on an individual's exceptional circumstance rather than general policy. Students have no way to distinguish a verified, authoritative answer from a confident but wrong one.

The verification gap has the following compounding dimensions:

- Temporal decay: Programme policies change across cohorts. An answer correct for VINS 2025 may be incorrect for VINS 2026. There is no mechanism to flag or update stale answers.
- Context specificity: Some answers are personal (e.g., 'in my case, the TA said...') rather than general policy. Without curation, personal edge cases masquerade as authoritative guidance.

2.2.3. The Reopening Problem

Even after an answer is accepted, the programme may change, or the answer may be discovered to be wrong. Traditional FAQ systems have no mechanism to handle this — once an answer is published, it stays, regardless of accuracy. This creates a trust erosion problem: as stale answers accumulate, students stop trusting the FAQ system entirely, reverting to informal channels and recreating the primary problem.

2.3. Objectives & Design Requirements

2.3.1. Primary Objective

Design and deploy a real-time, fault-tolerant web application that:

1. Proactively deflects duplicate questions — before the student submits — by surfacing existing answers during the question composition workflow.
2. Enforces a mandatory administrative approval gate before any peer answer is promoted to a Canonical FAQ, ensuring accuracy and policy alignment.

3. Enables community correction of wrong answers, allowing incorrect verified answers to be flagged and returned to the peer-review queue for re-resolution.
4. Retains all resolved knowledge permanently, making it searchable and accessible to future cohorts without administrative re-effort.

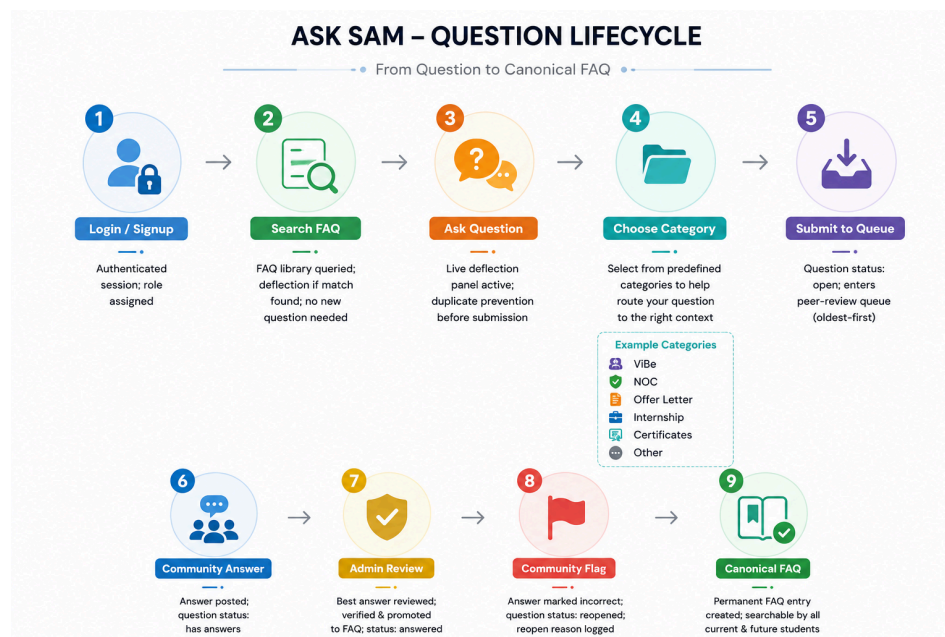
2.3.2. Functional Requirements

- Fault tolerance: System must serve read-only FAQ content even during database outages, preventing total information blackouts during peak cohort periods.
- Frictionless UX: An unauthenticated student composing a question must not lose their typed text on redirect to login — the system must preserve and auto-post after authentication.
- Zero knowledge loss: Once an answer is admin-verified and promoted to FAQ, it must persist permanently and be indexed for search by all future cohort members.
- Performance: Smart search must respond within 300ms of keystroke to be effective as a deflection mechanism.
- Scalability: The architecture must accommodate growing cohort sizes without architectural changes, using horizontally scalable components (MongoDB Atlas, NestJS, Vite SPA).

2.4. Platform Workflow:

2.4.1. The Data Lifecycle

AskSam imposes a strict lifecycle on every student question, ensuring that only verified, accurate information reaches the permanent FAQ library:



2.4.2. Why Each Stage Matters

Stage 2 & 3 — Search and Deflection are the most impactful stages for reducing administrative load. By surfacing existing answers at the moment of intent, before a question is submitted, AskSam eliminates the problem at the source. The live deflection panel during question composition is the key UX innovation that operationalises this.

Stage 6 — Admin Verification is the trust anchor of the entire system. By requiring explicit administrator approval before any answer graduates to a Canonical FAQ, AskSam prevents the propagation of incorrect peer answers. The admin acts as an editorial gate, not a bottleneck — most of the intellectual work is done by the peer community; the admin only confirms.

Stage 7 & 8 — Reopen Flow solves the temporal decay problem. Because policies change and answers can be wrong, AskSam provides a community-powered correction mechanism. Any student can flag a verified answer as incorrect; the question returns to the peer-review queue with a logged reopen reason, and the cycle restarts. This makes the knowledge base self-correcting over time rather than progressively stale.

2.5. Technical Approach & Architecture

2.5.1. Stack Selection Rationale

Every component of the AskSam stack was chosen to directly serve the problem requirements:

Technology	Role	Problem It Solves
React 18.3 + Vite 5.3	Frontend SPA	Fast, code-split, lazy-loaded pages; frictionless student UX
Tailwind CSS v4	Styling	Consistent design system; CSS-native animations without bundle bloat
TanStack Query v5	Server state	Cache-first fetching; stale-while-revalidate for responsive UI
react-quill-new	Rich text editor	Formatted answers with code blocks and image embeds
NestJS 10.4	Backend API	Typed, modular, guard-based; scales to large cohorts cleanly
Mongoose 8.5 + MongoDB	Database	Flexible schema for evolving FAQ/Question/Answer documents
JWT + RolesGuard	Auth & Authz	Stateless, scalable; strict admin/user privilege separation
Socket.IO	Real-time	Live queue updates; instant notifications to question askers
faqData.json fallback	Fault tolerance	Read-only mode survives MongoDB outages; zero blackout risk
@nestjs/throttler	Rate limiting	Protects auth endpoints from spam/brute-force

2.5.2. Expected Impact & Success Metrics

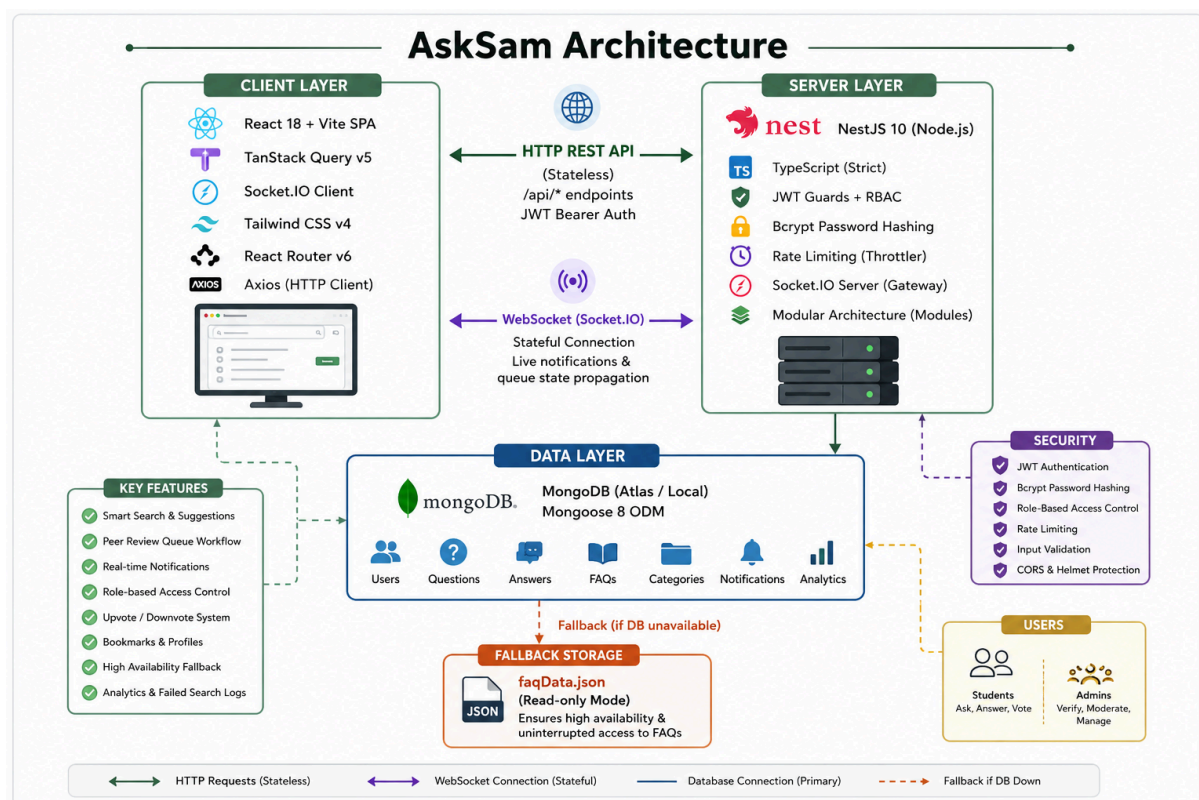
Metric	Baseline (<u>Pre-AskSam</u>)	Target (<u>Post-AskSam</u>)
Duplicate questions per cohort	High (untracked; estimated 60-70% of TA queries)	< 20% of queries reach TAs (rest deflected by search)
Time-to-answer for policy questions	Hours to days (depends on TA availability)	< 5 minutes (self-service FAQ search)
Incorrect peer answers in circulation	Uncontrolled (no verification gate)	0 unverified answers in canonical FAQ
Knowledge retention across cohorts	0% (ephemeral channels)	100% (all verified answers permanently indexed)
Failed searches logged & actioned	Not tracked	Weekly admin review; new FAQs created within 48 hours

3.SYSTEM ARCHITECTURE AND DATABASE UNDERSTANDING

This section provides a detailed technical overview of AskSam's architectural design and its underlying data models. It is intended to serve as the definitive reference for the Database Architecture workstream and for any developer or evaluator seeking to understand how data flows through the platform.

3.1 Architecture Overview

AskSam follows a classic Client-Server architecture, where a React single-page application (SPA) running in the browser communicates with a NestJS API running on the server. The two layers are deliberately kept separate — the frontend never touches the database directly. All data access is mediated through the server, which enforces authentication, authorization, and business logic.



3.1.1 The Stateless REST API

All primary data operations — submitting questions, fetching FAQs, voting, managing user accounts — are performed over a stateless REST API. The API is prefixed under /api and uses standard HTTP verbs (GET, POST, PATCH, DELETE).

Stateless means the server holds no session memory between requests. Instead, every protected API call must carry a JSON Web Token (JWT) in the Authorization: Bearer <token> header. The NestJS server validates this token on every single request using Guard decorators, then either processes the request or rejects it with an HTTP 401 Unauthorized error.

This design has two important benefits. First, it is horizontally scalable — any number of server instances can handle requests without sharing session state. Second, it is secure by default, because compromised tokens can be revoked at the JWT secret level.

Key Point: JWT tokens are signed using the JWT_SECRET defined in the backend .env file. Every token carries the user's ID and role (user / admin), which drives role-based access control across all protected endpoints.

3.1.2 The Stateful WebSocket Connection

Not all interactions in AskSam can wait for the user to refresh the page. When a question gets answered, an upvote is received, or an admin verifies a submission, the involved users must be notified immediately. This is achieved via a persistent, stateful WebSocket connection managed by [Socket.IO](#).

Unlike the REST API, WebSocket connections are long-lived. Once a logged-in user opens AskSam in their browser, a bidirectional channel is established between the client and the NestJS WebSocket Gateway. This channel stays open until the user closes the tab.

Two types of real-time events are broadcast over this channel:

- Targeted notifications — delivered only to the specific user whose content was interacted with (e.g., "Your question was answered by a peer").
- Broadcast state updates — delivered to all connected clients viewing the moderation queue, instantly removing a question from the list the moment another user submits an answer, preventing duplicate effort.

3.2 Database Models (MongoDB Collections)

AskSam uses MongoDB as its primary document store, accessed via the Mongoose 8 ODM (Object-Document Mapper). MongoDB's flexible, schema-less nature makes it well-suited for a Q&A platform where questions, answers, and metadata can vary widely in structure.

The platform is organized into six core collections. Each collection maps to a Mongoose Schema file in the backend/src/schemas/ directory.

Overview of all collections:

3.2.1 User Schema (user.schema.ts)

Schema File	MongoDB Collection	Primary Responsibility
user.schema.ts	users	Authentication, profiles, roles, bookmarks
question.schema.ts	questions	Student submissions and their review lifecycle
faq.schema.ts	faqs	Permanently verified, canonical knowledge entries
category.schema.ts	categories	Dynamic content tracks managed by admins
notification.schema.ts	notifications	Real-time event triggers for Socket.IO
search-analytics.schema.ts	searchanalytics	Logs of zero-result searches for gap analysis

The User document is the foundation of AskSam's authentication and participation model. Every interaction — submitting questions, posting answers, bookmarking FAQs — is tied to a User document by its MongoDB ObjectId.

Field	Type	Description
fullName	String (required)	The student's full display name shown in the UI.
username	String (unique)	A unique handle used for @mentions and profile URLs.
email	String (unique)	Primary login credential. Indexed for fast lookups.
password	String (hashed)	The student's password, hashed with Bcrypt before storage. Plain text is never persisted.

role	Enum: user admin	Controls permission level. Defaults to 'user'. Only 'admin' can approve categories, convert answers to FAQs, and access the admin dashboard.
reputation	Number	A cumulative score reflecting contribution quality — incremented by upvotes received, answers verified, etc.
savedFaqs	ObjectId[] (ref FAQ)	Array of FAQ document IDs representing the user's personal bookmark list.
contributionLog	Date[]	An array of timestamps recording each day the user posted an answer, used to generate the GitHub-style contribution heatmap.

Role-Based Access Control (RBAC)

The role field is the pivot of AskSam's access control system. NestJS Guards inspect the JWT payload on every incoming request and extract the role claim. Routes decorated with `@Roles('admin')` will automatically return HTTP 403 Forbidden to any request made by a standard user, regardless of whether they hold a valid token.

In practice, this means:

- Students (role: user) can search FAQs, submit questions, post answers, vote, bookmark, and flag incorrect answers.
- Administrators (role: admin) can additionally approve category tracks, convert peer-reviewed answers into canonical FAQs, and access the failed-search analytics dashboard.

Security Note: Bcrypt hashes stored in the password field are one-way — the original password cannot be recovered. During login, NestJS compares the submitted password against the stored hash using `bcrypt.compare()`, never storing or transmitting the plain-text value.

3.2.2 Question Schema (question.schema.ts)

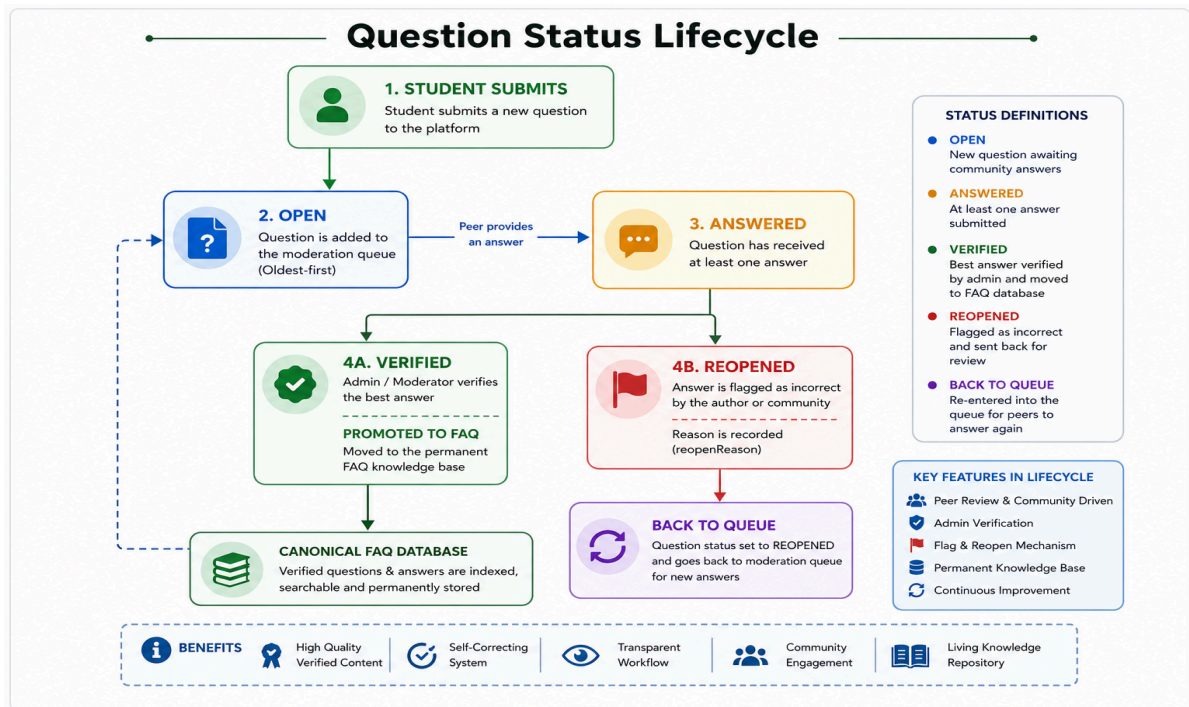
The Question document represents a student's submitted query and is the central data artifact of AskSam's collaborative workflow. Every question passes through a defined status lifecycle from the moment it is created until it is either promoted to a canonical FAQ or corrected via the reopen flow.

Field	Type	Description
title	String (required)	A concise summary of the question, displayed in search results and the moderation queue.
description	String	The full question body, supporting rich-text HTML from the React Quill editor (bolding, lists, code blocks, image URLs).
author	ObjectId (ref User)	Reference to the User who submitted the question. Used to send Socket.IO notifications and display attribution.
status	Enum (see below)	The current lifecycle stage of the question. This field drives the entire moderation workflow.
answers	ObjectId[] (ref Ans.)	Array of community-submitted answer document references, ordered by upvote count.
upvotes	Number	Total count of positive community votes on the question itself.
downvotes	Number	Total count of negative community votes.

views	Number	Incremented each time the question thread page is loaded.
tags	String[]	Free-form labels attached during submission for filtering and visual badge display.
category	ObjectId (ref Cat.)	Links the question to a structural content track (e.g., ViBe, NOC).
reopenReason	String	Populated when a user flags a verified answer as incorrect. Records the reason for re-routing the question back to the queue.
createdAt	Date (auto)	Timestamp set on creation. The oldest-first queue algorithm sorts by this field to prevent stale questions from being ignored.

The Status Lifecycle

The status field is the single most important field in the Question schema. It acts as a state machine that controls where a question appears in the UI, who can interact with it, and what actions are available.



Status Value	Meaning	Where It Appears
open	Question submitted, awaiting a peer answer.	Moderation Queue (sorted oldest-first).
answered	At least one peer answer has been submitted. Awaiting admin review.	Thread view; admin dashboard for verification.
verified	Admin has approved a best answer and converted it to an FAQ.	Removed from queue; promoted to FAQ collection.
reopened	A user flagged the answer as incorrect, providing a reopenReason.	Re-inserted into the moderation queue for re-answering.

Design Rationale: The oldest-first queue algorithm (sorting by createdAt ASC) is a deliberate choice to ensure fairness. In a typical Q&A system, newer questions tend to attract attention, leaving older ones permanently unanswered. AskSam's queue surfaces the most neglected question first, guaranteeing that every submission eventually receives a response.

3.2.3 FAQ Schema (faq.schema.ts)

The FAQ collection is AskSam's permanent knowledge treasury — the definitive, curated library of verified question-and-answer pairs. A document enters this collection only via an explicit admin action: the "Convert to FAQ" operation, which extracts the verified answer from a Question document and creates a new, immutable FAQ entry.

This separation between the transient Question collection (which tracks the review workflow) and the persistent FAQ collection (which stores stable knowledge) is a core architectural decision. It means the FAQ library never contains unverified or in-progress content.

Promotion Flow: When an admin clicks "Convert to FAQ", the NestJS API (1) creates a new FAQ document with the question, verifiedAnswer, and category; (2) sets the source Question's status to 'verified'; and (3) broadcasts a Socket.IO notification to the original question author. The Question document is preserved for audit purposes; only a copy is moved to the FAQ collection.

3.2.4 Category Schema (category.schema.ts)

Categories are the structural backbone of AskSam's content organization. They represent named knowledge tracks — for example, ViBe (Vikram Betal), NOC (No Objection Certificates), and Internships — that allow students to browse and filter FAQs by topic without needing to know specific search terms.

The Category system is dynamic: any user can propose a new category track, but it does not appear in the public UI until an admin explicitly approves it. This prevents category sprawl and maintains a clean, navigable taxonomy.

Field	Type	Description
name	String (required)	The human-readable label displayed in the UI (e.g., "ViBe", "NOC", "Admissions").
slug	String (unique)	A URL-safe identifier derived from the name (e.g., vibe, noc). Used in browser URLs for filtered FAQ views.
description	String	A brief explanation of what topics this category covers, shown in the category index.

isApproved	Boolean	The approval gate. Only categories where isApproved: true are shown in the public navigation and search filters. Controlled exclusively by admins.
createdBy	ObjectId (ref User)	Reference to the User who proposed the category, used for accountability.
faqs	ObjectId[] (ref FAQ)	Array of FAQ document IDs belonging to this category. Used to count and display category size.

The isApproved approval gate is enforced at the API level using NestJS Guards. The PATCH /api/categories/:id/approve endpoint is decorated with @Roles('admin') — any attempt to call this endpoint without an admin JWT will receive a 403 Forbidden response immediately, without the request ever reaching the controller logic.

3.2.5 Notification Schema (notification.schema.ts)

The Notification collection stores a persistent history of every real-time event generated on the platform. While Socket.IO handles the instant delivery of alerts, the Notification schema ensures users can revisit missed notifications — for example, after logging back in — via the /notifications page.

Field	Type	Description
recipient	ObjectId (ref User)	The User this notification is addressed to.
type	Enum String	The event type (e.g., 'answer_received', 'answer_verified', 'upvote_received', 'answer_flagged').
message	String	The human-readable notification text shown in the toast and notification center.
relatedId	ObjectId	A polymorphic reference to the relevant document (a Question or FAQ) for

		deep-linking from the notification.
isRead	Boolean	Tracks whether the user has seen this notification. Used to display the unread badge count on the notification bell icon.
createdAt	Date (auto)	Timestamp for ordering the notification history chronologically.

3.2.6 SearchAnalytics Schema (search-analytics.schema.ts)

The SearchAnalytics collection is a lightweight but strategically important logging mechanism. Every time a user performs a search query on AskSam and receives zero results — meaning the knowledge base has no matching FAQ or open question — the query is automatically recorded as a document in this collection.

The purpose is to build an evidence-based hit-list of knowledge gaps. Rather than guessing what content to create next, administrators can open the `/api/admin/failed-searches` endpoint to see exactly which questions students are asking that the platform cannot yet answer, and prioritize FAQ creation accordingly.

Field	Type	Description
query	String (required)	The exact search string that returned zero results. Stored verbatim for accuracy.
searchedBy	ObjectId (ref User)	Optional reference to the User who performed the search (null for guest searches). Helps identify patterns per cohort.
count	Number	Incremented each time the same query returns no results. High-count queries indicate the most urgent content gaps.
createdAt	Date (auto)	Timestamp of the first occurrence of this failed search.

lastSearched	Date	Timestamp of the most recent occurrence. Allows admins to distinguish stale gaps from currently active ones.
--------------	------	--

Admin Workflow: Admins visit the Failed Search Analytics dashboard to review the top zero-result queries sorted by count descending. They then create new FAQ entries or approve pending questions that match those gaps, completing a self-improving content loop.

3.3 Fault Tolerance: The `faqData.json` Offline Fallback

One of AskSam's standout engineering decisions is its approach to database unavailability. Standard student projects — and even many production systems — will simply crash and return an HTTP 500 error if their database connection drops. AskSam is designed differently: it degrades gracefully.

3.3.1 How It Works

The NestJS application startup sequence includes a database connectivity check. If the connection to the MongoDB cluster (whether a local instance or a MongoDB Atlas cloud cluster) cannot be established, the following fallback chain activates automatically:

- The NestJS service layer detects the failed Mongoose connection event.
- Instead of crashing the process, the server switches into read-only mode.
- All GET requests to `/api/faqs` are re-routed to read from a static `faqData.json` file bundled within the backend directory.
- All write operations (question submissions, answers, votes) return a user-facing error message indicating that the system is temporarily in read-only mode, rather than an opaque 500 error.

3.3.2 Why This Matters

For a platform used by students during high-stress periods — assignment deadlines, exam seasons — availability is critical. The `faqData.json` file is designed to hold a curated snapshot of the most important, frequently accessed FAQs. This means the core utility of the platform (looking up answers to common questions) remains available even during infrastructure outages.

Mode	Trigger	Available Operations	Unavailable Operations
Normal (Online)	MongoDB connected successfully	All read and write operations	None
Fallback (Offline)	MongoDB connection failed or timed out	Read FAQs from <code>faqData.json</code>	Submit questions, post answers, vote, authenticate new users

Maintenance Note: The `faqData.json` file must be periodically updated by the platform maintainers to reflect the current state of the live FAQ library. A recommended practice is to run an export script after any significant batch of FAQ promotions to keep the fallback snapshot current.

3.3.3 Implementation Location

The fallback logic is implemented in the FAQ service module at `backend/src/modules/faq/faq.service.ts`. The static JSON file is located at `faqData.json` and is read into memory once at startup when fallback mode is triggered, avoiding repeated disk I/O on every request.

3.4 Section Summary

The following table consolidates the key architectural and database decisions described in this section for quick reference:

Design Decision	Implementation	Benefit
Client-Server separation	React SPA ↔ NestJS REST API	Security, independent scaling, clear boundary of concerns
Stateless authentication	JWT Bearer tokens in every request	Horizontal scalability; no server-side session state
Real-time communication	Socket.IO WebSocket gateway	Instant notifications; live queue synchronization
Role-based access control	JWT role claim + NestJS <code>@Roles()</code> Guard	Enforced at framework level; cannot be bypassed

Document database	MongoDB via Mongoose 8 ODM	Flexible schema for evolving Q&A data structures
Status-driven workflow	Question.status enum state machine	Clear audit trail; controls moderation queue visibility
Knowledge promotion	Separate Question and FAQ collections	Permanent knowledge base is always clean and verified
Self-improving content	SearchAnalytics zero-result logging	Data-driven FAQ creation; closes knowledge gaps systematically
Graceful degradation	faqData.json offline fallback	Platform remains usable during database outages

4. METHODOLOGY AND PLATFORM WORKFLOWS

This section traces the logical journey a user takes through AskSam — from first login to the permanent promotion of a peer-verified answer into the canonical knowledge base. Each workflow is described from the user's perspective, with the underlying technical mechanisms explained where they produce visible or meaningful effects.

4.1 The Auth Flow

AskSam's authentication is built around three principles: security, statefulness, and zero friction for the user. These are delivered through three complementary mechanisms that work in sequence.

Bcrypt Password Hashing

When a user registers, their plaintext password is never stored. It is passed through Bcrypt, a one-way hashing algorithm that deliberately runs slowly — via a configurable cost factor — to resist brute-force attacks. The resulting hash string is what gets saved to MongoDB. On login, Bcrypt re-hashes the submitted password and compares it against the stored hash. The original password is never recoverable, even by a database administrator.

Security note: *Bcrypt's deliberate slowness means that even if the database were compromised, an attacker would face an infeasible computation cost to reverse any individual hash.*

JWT & HTTP-Only Cookies

JWT & Local Storage Upon successful login, the NestJS backend mints a JSON Web Token (JWT) — a cryptographically signed, self-contained string that encodes the user's ID and role. The React frontend securely stores this token in browser localStorage. Each subsequent API request automatically attaches this token as a Bearer header via an Axios interceptor, and the NestJS JWT Guard validates the signature before allowing access to any protected route.

The Frictionless Login Modal

This is the most user-experience-critical feature of the auth system. A common failure mode in web applications is the redirect wall: a user types out a detailed, thoughtful question and clicks submit, only to be bounced to a login page — and their typed content is gone.

AskSam resolves this with a modal overlay. The user's typed content is held in React component state. The login form slides over the current screen without navigating away. Upon successful authentication, the saved query is posted immediately and automatically. Not a single keystroke is lost.

Auth flow at a glance

1	User submits a question React checks authenticated state before calling the API.
2	Not logged in? Text is saved in component state. Modal overlay appears.
3	Bcrypt verification Submitted password is hashed and compared against the stored hash.
4	JWT issued A signed token is set as an HTTP-only cookie. JavaScript cannot read it.
5	Auto-post on success The previously saved question text is posted immediately.

4.2 The Search & Deflection Flow

This flow is designed around a single insight: the best duplicate question is one that never gets submitted. AskSam implements two layers of deflection that activate at different points in the user's journey.

Layer 1 — Predictive Smart Search

As the user types into the main search bar, the frontend sends debounced queries to GET /api/faqs on the backend. Results populate a real-time dropdown below the input, showing existing verified answers. The user may find their answer before they even finish typing — this is the primary deflection mechanism. No submission, no queue entry, no moderation effort required.

Layer 2 — The Live Deflection Panel

This is the secondary, more assertive mechanism that activates specifically on the question submission page. As a user types the body of a new question, a sidebar panel simultaneously queries the database for semantically similar existing questions or FAQs. If matches are found, they are displayed live alongside the composition area — the relevant existing content is shown while the user is still writing.

Only if no suitable answer is surfaced does the user proceed to submit their question. This two-layer system — search deflection first, composition-time deflection second — significantly reduces queue noise before any moderation effort is required.

Design rationale: *Deflection at the point of composition is significantly more effective than deflection after submission. A user who sees their answer while typing has no reason to create a duplicate.*

4.3 The Moderation Queue Flow

The queue at GET /api/questions/queue is the collaborative working layer of AskSam. It surfaces all questions with a status of open or reopened that have not yet been resolved by a verified answer.

Oldest-First Sorting Algorithm

Questions are sorted chronologically ascending, ensuring the queue is drained in first-in, first-out order. This is an explicit fairness mechanism. Without it, newer or more engaging questions would attract peer attention while older unanswered questions stagnated indefinitely. The oldest-first algorithm guarantees that every submitted question is eventually resolved, regardless of topic popularity.

Collaborative Resolution via Real-Time State

Any authenticated peer can open a question from the queue and submit an answer. Once answered, the question's status transitions and it is removed from the queue view. Critically, this state change is propagated in real-time to all connected clients via Socket.IO. If two peers are looking at the same question simultaneously and one answers it, it immediately disappears from the other's screen — preventing duplicated effort across the community.

Question status lifecycle

Status	Description
open	Newly submitted question. Enters the queue sorted by creation date (oldest first).
answered	A peer has submitted a response. Awaiting admin verification before promotion.
reopened	A verified answer was flagged as incorrect. A reopenReason is logged and the question returns to the queue.
FAQ	Admin has verified the answer and promoted it to the canonical knowledge base.

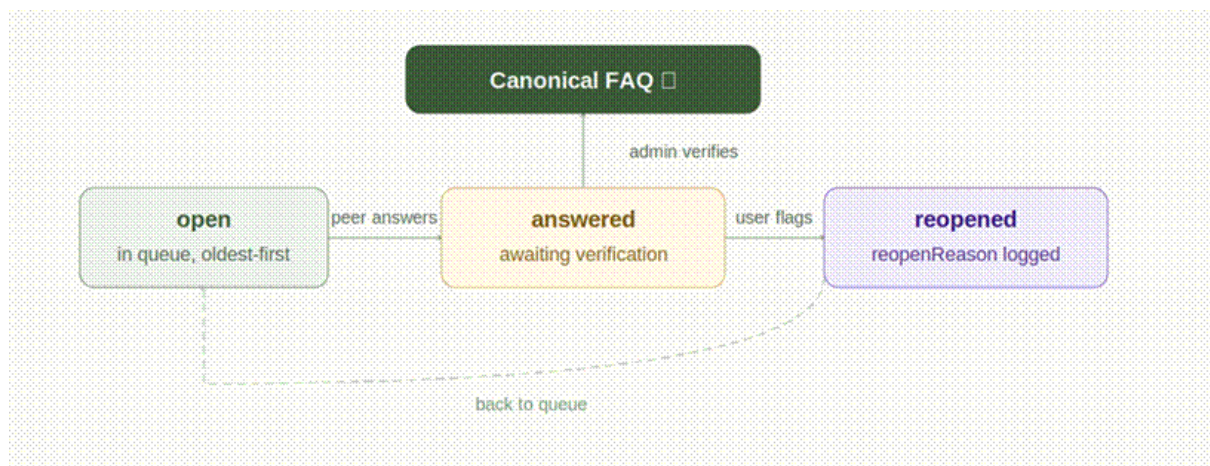


Figure — Question lifecycle: from submission to canonical FAQ or correction via the reopen flow.

4.4 The Answer & Voting Flow

Once a user opens a question thread from the queue, they are presented with a full composition environment designed to encourage high-quality, well-formatted responses.

React Quill Rich-Text Editor

Rather than a plain text area, answers are composed in a React Quill editor instance that supports bold, italics, inline code blocks, and ordered and unordered lists. This is significant for an academic platform: a peer can share a precise algorithm with a fenced code block, or

format a numbered step-by-step process, rather than pasting unstructured text. The resulting content is stored as HTML in MongoDB and rendered faithfully in the thread view.

Media Embedding

Users can paste external image URLs directly into the answer body to embed visual context — a screenshot of an error message, a diagram from documentation, or a photograph of a physical process. This is handled by Quill's image handler and stored as a URL reference rather than as a binary blob, keeping the database lean while preserving rich visual context.

Community Voting

Every submitted answer carries upvote and downvote controls, mapped to `PATCH /api/questions/:id/vote` on the backend. This surfaces the highest-quality answer organically: when a question accumulates several peer responses, the community's collective signal pushes the most accurate and well-explained answer to the top.

An admin reviewing a thread for verification can immediately see which answer the community trusts most — the voted ranking serves as a quality pre-sort, reducing the cognitive load of the verification decision.

4.5 The Admin Verification Flow

This is the most consequential action in the entire platform — the moment a transient peer answer becomes permanent institutional knowledge.

The "Convert to FAQ" Action

When an admin reviews a question thread and determines that a peer's answer is correct and comprehensive, they trigger `POST /api/faqs/convert`. The NestJS service then executes the following sequence:

1	Read source documents The Question document and its selected answer are retrieved from MongoDB.
2	Create FAQ document

	A new FAQ record is created, populating question, answer, categoryId, and metadata from the source question.
3	Promote to canonical library The new FAQ is immediately searchable at GET /api/faqs by all users.
4	Update question status The original Question document is marked as resolved.
5	Socket.IO notification The question's author receives a live toast notification that their question has been verified.

Why the Two-Step Model Matters

A direct-post model — where any user's answer immediately becomes a FAQ — would pollute the knowledge base with unverified, potentially incorrect information. The peer-review queue acts as a draft layer; admin verification is the editorial gate. The community provides content at scale; a trusted reviewer provides the quality guarantee. Together they create a self-sustaining knowledge base that grows without degrading in accuracy.

The Reopen Escape Valve

Verification is not irreversible. If a promoted FAQ is later found to be incorrect, any user can flag it with a reopenReason. This action:

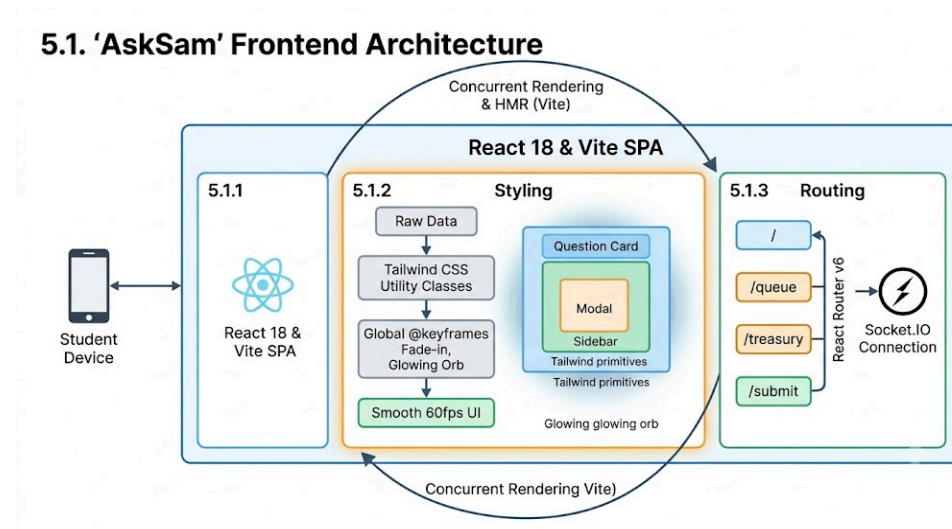
- flips the question's status back to reopened,
- logs the stated reason for accountability and audit,
- returns the question to the moderation queue for a new cycle of peer answers and admin verification.

The data lifecycle is self-correcting by design. No answer is permanently locked in as correct — the community retains the ability to challenge and revise the knowledge base as understanding evolves.

Summary: *Search deflects duplicates. The queue surfaces unresolved questions fairly. Rich editing raises answer quality. Voting surfaces the best answer. Admin verification gates promotion. The reopen flow corrects errors. Each mechanism feeds the next.*

5. IMPLEMENTATION DETAILS AND TECH STACK

5.1 Frontend (Client-Side)



5.1.1 Framework: React 18 & Vite

The client application is built with React 18, leveraging its concurrent rendering model to keep the UI responsive even during high-frequency real-time updates — a necessity in a live event like Samagama where dozens of students may interact with the queue simultaneously. The project is scaffolded and bundled using Vite, which replaces traditional bundlers like Webpack with a native ES module development server. This gives the team near-instant Hot Module Replacement (HMR) during development and significantly faster cold-start times, making iteration on UI components efficient throughout the build cycle.

5.1.2 Styling: Tailwind CSS v4 with Custom CSS Animations

All UI styling is managed through Tailwind CSS v4, using its utility-first class system to keep component styles self-contained and easy to scan. No external component library is imported — every element from cards to modals is composed from Tailwind primitives, keeping the bundle size minimal and the design system fully in the team's control.

Rather than pulling in heavy JavaScript animation libraries such as Framer Motion or GSAP, all motion effects are implemented as custom CSS `@keyframes` animations declared in the global stylesheet. This includes:

Fade-in transitions applied to question cards, modals, and route-level elements as they mount into the DOM, giving the interface a polished, intentional feel without JavaScript overhead.

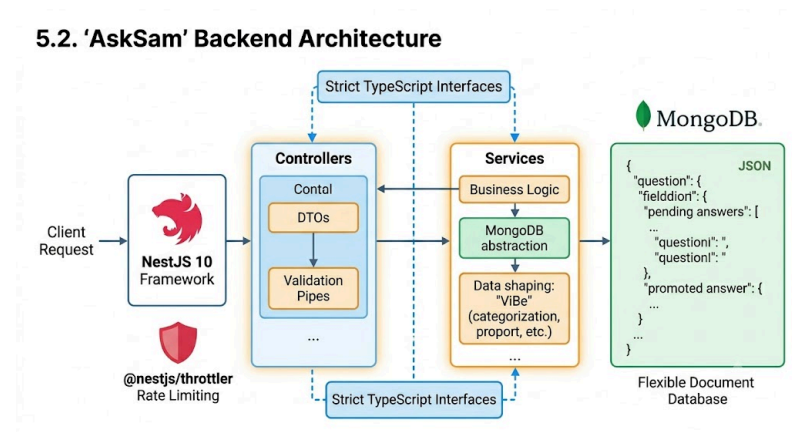
Glowing orb effects used as ambient background decorations — layered radial gradients animated with cycling opacity and transform keyframes — that give the AskSam interface its distinctive visual identity without impacting runtime performance.

This approach keeps the JavaScript bundle lean and ensures smooth 60fps rendering even on lower-end student devices.

5.1.3 Routing: React Router v6

Client-side navigation is implemented with React Router v6, enabling a true Single Page Application (SPA) architecture where all route transitions happen in-browser without full page reloads. This is particularly important for AskSam because it preserves the active Socket.IO WebSocket connection across page navigation. Routes are declared using the `Routes` and `Route` component API. Programmatic redirects use the `useNavigate` hook, keeping navigation logic close to the components that trigger it.

5.2 Backend (Server-Side)



5.2.1 Framework: NestJS 10

The server is built on NestJS 10, a progressive Node.js framework that enforces a modular, opinionated architecture out of the box — a deliberate choice for a project built collaboratively across a student lab, where consistent code structure matters as much as functionality. The codebase strictly follows NestJS's three-layer design pattern:

Controllers are responsible solely for handling incoming HTTP requests and delegating to the appropriate service. They contain no business logic — their role is to parse request parameters, apply validation pipes, and return the shaped response.

Services contain all business logic: fetching questions from MongoDB, promoting a peer-reviewed answer to the knowledge treasury, categorizing entries under structural tracks like ViBe (Vikram Betal), and computing queue state.

Strict TypeScript Interfaces are enforced across the entire request lifecycle. Every incoming payload is typed through a DTO (Data Transfer Object), every service method carries typed input and return signatures, and every database document shape is described by an interface.

5.2.2 Database: MongoDB

Persistent data — questions, answers, user records, category metadata, and the knowledge treasury — is stored in MongoDB. Its document model maps naturally onto the nested, schema-flexible structure of FAQ content, where a question might carry zero answers, one pending peer-reviewed response, or a promoted definitive answer, without requiring schema migrations as the data model evolves.

5.2.3 Security: API Rate Limiting with `@nestjs/throttler`

To prevent spam submissions and protect the question queue from abuse, the backend applies `@nestjs/throttler` for request rate limiting. The throttler guard is configured globally so that every endpoint benefits from protection without requiring per-route annotation. Any client that exceeds the configured request threshold within the time window receives a 429 Too Many Requests response and is temporarily blocked.

5.3 Real-Time Communication: Socket.IO (WebSockets)

5.3.1 Instant Toast Notifications

When a peer posts a new answer to a question, or when a verified peer promotes a response to the permanent knowledge treasury, the server immediately emits a targeted Socket.IO event to the relevant subscribed clients. The frontend listens for this event and renders a toast notification — a brief, non-blocking overlay that appears at the edge of the screen — without any user-initiated action or page refresh.

5.3.2 Live Queue State Propagation

When a question card in the pending queue is answered and verified, it must disappear from the queue for every connected user at the same instant. Socket.IO solves this through a broadcast event emitted by the server the moment a question's status changes. Every connected client receives the event and reactively removes the corresponding card from their local UI state, keeping the live queue view perfectly synchronized across all sessions with no refresh required.

To provide a live collaborative experience, AskSam integrates Socket.IO, enabling real-time communication between the server and connected clients.

6. RESULTS AND DISCUSSIONS

Performance results

Component	Result
Vite Frontend	Provided responsive page rendering and smooth navigation across platform modules.
NestJS Backend	Offered a structured backend architecture for handling authentication, questions, FAQs, notifications, and moderation workflows.
Socket.IO Integration	Enabled real-time notifications and platform updates.
MongoDB Database	Efficiently managed user, question, FAQ, notification, and category data.
Offline FAQ Fallback	Ensured FAQ accessibility through the fallback JSON mechanism when required.

UX/UI Successes

Feature	Success Achieved
Dual-Tab Login Interface	Simplified user authentication and account creation.
Notification Center	Helped users stay updated with platform activities and responses.
Interactive Modals	Reduced unnecessary page navigation and improved usability.
Predictive Search & Live Deflection Panel	Assisted users in finding existing answers before posting duplicate questions.
Responsive Design	Improved accessibility across different screen sizes and devices.
Rich Text Editor	Enabled user to provide detailed and structured answers

Challenges Faced

Challenge	Resolution
Understanding the complete workflow of questions, answers, verification, and FAQs	Studied the platform architecture and mapped the complete FAQ lifecycle before implementation.
Integrating multiple platform modules	Ensured smooth interaction between frontend, backend, database and notification system
Managing role-based workflows	Implemented clear separation between student and administrator functionalities.
Designing a user-friendly interface	Improved navigation, responsiveness, and usability through continuous refinement.
Coordinating various platform features	Successfully integrated authentication, moderation, notifications, search, and FAQ management into a unified system.

DISCUSSION

The AskSam platform successfully addressed the problem of repetitive academic and administrative queries by providing a centralized, structured, and verified knowledge-sharing system. Features such as predictive search, FAQ verification, role-based moderation, and real-time notifications contributed to improving information accessibility and reliability. The project also provided practical experience in full-stack web development, database management, authentication mechanisms, and collaborative software development.

7. CONCLUSION

The AskSam platform delivers a highly effective, scalable solution to the administrative and informational challenges of an expanding academic internship program. By combining real-time deflection panels with a rigorous administrative verification workflow, the platform successfully balances community-driven knowledge sharing with institutional accuracy.

Architecturally, AskSam leverages a robust full-stack foundation—integrating React, NestJS, and MongoDB with Socket.IO for real-time responsiveness and JWT for stateless security. The inclusion of reputation tracking and an offline fallback ensures a reliable, production-ready user experience that naturally incentivizes high-quality participation.

Ultimately, AskSam transforms a high-volume stream of repetitive, unmanaged queries into a structured, self-reinforcing knowledge ecosystem. Every interaction—whether a question deflected, an answer verified, or an out-of-date response flagged—actively builds a permanent institutional memory for the Samagama community. With a clear architectural pathway toward mobile support, email integrations, and cross-batch knowledge transfer, AskSam is well-positioned to scale alongside the program, reducing administrative overhead while continuously elevating the student experience for cohorts to come.

APPENDIX

- * A. UI Screenshots: Provide screenshots of the Home Page, the Ask Workflow, the Login Modal, and the Admin Dashboard.
- * B. System Diagrams: Insert the Mermaid Architecture diagram from the README.
- * C. API Endpoints List: Paste the categorized REST API routes (e.g., GET /api/questions/queue, POST /api/faqs/convert).